

DA-HERO

DatawiseAgent Harness-Engineering for Resource Optimization
让 DatawiseAgent 的任务适配、长交互和资源预算可控

汇报人：於之翔，组员：冉子易、张恒

2026.6.10

复现以后，三条值得优化的线

方向	核心问题
1. 任务适配	不同 benchmark 的失败源并不一致。InfiAgentBench 主要暴露答案名、统计定义、预处理顺序和最终格式不稳定；DataModeling 则更多暴露 submission 合法但预测低信息、metric 和 manifest 不完整等问题。因此 harness 不能只做统一 prompt，而要先识别任务类型和错误源。
2. 结构化时间换性能	长交互只有在每一轮都有明确检查目标时才有意义。我们需要把 planning、执行、debug、reformat 拆成可验证环节，失败时只修失败点，并用 final block / verifier 避免从长日志中漏抽或错抽答案。
3. 资源感知调度	强模型并不一定更慢，小模型也不一定更省。资源优化的核心是把确定性 schema、Answer Spec、format checker 前置，把强模型留给高风险推理和复杂代码生成，同时减少重复探索和无效对话。

Takeaway: 三条线不是堆模块：先识别任务错法，再把交互结构化，最后按风险决定模型、记忆和预算。

Part I

Task-Specific Adaptation

从失败源出发，让 harness 按任务类型约束、验证和回收上下文

Problem: 两个 benchmark 的失败源并不一样

InfiAgentBench: 口径与过程漂移

复现分析: 代码通常能跑完, 但过滤/预处理顺序、统计定义、样本口径、答案名和最终格式会漂移; 错误不报错, 却影响闭式评分。

本质问题: 缺少稳定的任务约束和过程校验; 题面中的 Answer Contract 没有持续绑定到每一步计算、验证和最终输出。

典型错误

- 过滤、预处理或特征构造顺序不当;
- 年份边界、过滤条件、样本口径偏移;
- std / correlation / feature baseline 定义不稳;
- answer name / final format 不一致。

DataModeling: 长链路执行不稳定

复现分析: 运行链条长、环境依赖复杂、执行时间长, 会话/内核容易中断; 完成率先受工程链路影响, 合法提交后才暴露预测质量。

本质问题: 它不是单点答案抽取, 而是端到端建模流程控制: 数据理解、训练、验证、提交、manifest 和评估证据必须跨步骤一致。

典型错误

- 路径、依赖、会话中断导致 final 文件缺失;
- sample copy / fallback 被当成 final;
- 多分类塌缩、均值回归、低信息预测;
- metric / manifest / validation evidence 不完整。

设计指向: 闭式答案任务绑定任务约束; 长链路建模任务管理执行、质量证据与恢复机制。

Design I: Task-Aware Routing, 把隐性要求显式化

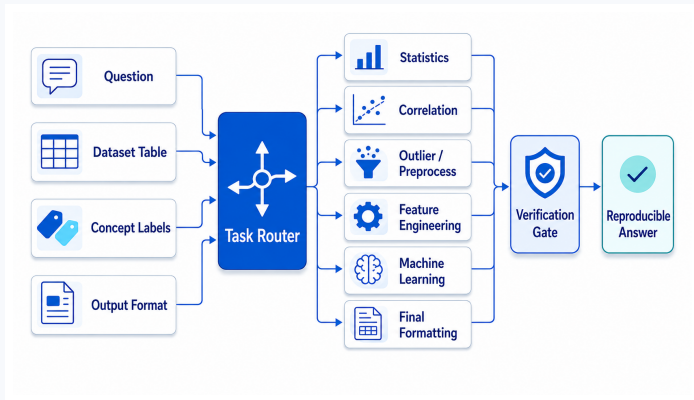
从题面抽取路由信号

- **level / concepts**: 难度与知识类型;
- **question / constraints**: 过滤、边界、方法;
- **format**: 答案名、小数位、输出模板。

协议 + 验证

不同任务进入统计、相关、异常处理、特征工程、机器学习或最终格式 protocol; verifier / finalizer 再检查答案名、统计口径和输出格式。

收束: 题面要求 → 可执行协议 → 验证后输出。



Design II: Context Engineering, 把长交互压缩成可恢复状态

三层上下文，不混在一个长 prompt 里

Transient Trace

单题 reason / action / observation, 只服务当前恢复, 不长期污染上下文

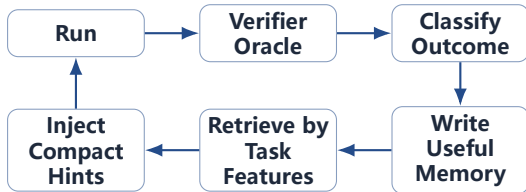
Curated Rule Cards

人工稳定规则: 统计口径、format、quality gate, 不自更新

Self-Updated Memory

agent 自己运行后的失败-修复经验、成功方法 trace, 按任务特征检索

借鉴 context compacting: 外部化状态 + 选择性回灌



关键原则

不能把 notebook 历史全量保存或回灌, 而要把长过程外部化成 **DataCard / Contract / Trace / Final Block / Manifest**; 后续只回灌与当前任务、错误和路由相关的片段。压缩、检索、遗忘共同决定后续推理质量。

Effect I: InfiAgentBench 上，口径与格式错误被显式收束

Setting	Split	ABQ	PASQ	UASQ
Qwen3-32B baseline	medium	83.72%	86.19%	86.00%
Task-Aware harness	medium	91.95%	93.06%	93.42%
Qwen3-32B baseline	hard	66.13%	78.49%	82.99%
Task-Aware harness	hard	78.41%	84.75%	87.19%

Answer Contract

答案名、目标字段和小数格式进入 FinalAnswerBlock，减少最终抽取和 reformat 漂移。

Method Check

统计口径、过滤边界和特征 baseline 由 rule cards / oracle 显式检查。

Hard Search

hard 题允许多分支解释和回退，但搜索成本需要交给后续 resource-aware 调度控制。

效果定位

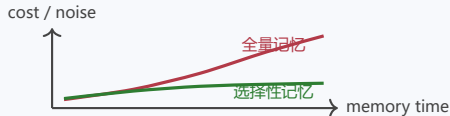
从“可跑的代码”进一步收束成“口径稳定、格式可验、证据可追”的答案通道。

Effect II: DataModeling 的边界与系统启发

Run	Valid submissions	Normalized Perf.	Avg time
Qwen2.5-72B baseline	68/74	0.4290	760.25s
Qwen3.5-35B-A3B strong ref.	73/74	0.5318	288.60s
Qwen3-VL-32B + harness	74/74	0.5173	297.49s

关键观察

- ▶ **74/74** 说明 harness 能稳定 completion;
- ▶ **0.5173** 接近强参考 0.5318, 但差距说明 quality gate 只能约束路径, 不能替代建模能力;
- ▶ 难点从“能否提交”转向 prediction quality、metric evidence 与任务族路由。



系统设计启发

- ▶ **Model-aware thickness:** 弱模型不能承载过厚流程; 强模型也需要保留搜索自由度, 只在高风险点加验证。
- ▶ **Selective memory:** 记忆价值取决于信噪比; 只沉淀 verifier / oracle 过滤后的失败-修复经验和成功方法 trace, 低价值、过期、冲突经验必须衰减或遗忘。
- ▶ **Training-free boundary:** harness 能约束路径、证据和恢复, 但不能替代强模型、后训练或领域建模经验。

Part II

Structured Time-for-Performance

多花时间不是多问几轮，而是把时间变成可验证过程

原作里有用的现象

弱模型可以靠 notebook-style 长交互补一部分能力，但有效部分不是“多聊天”，而是：

- ▶ 先形成可执行计划；
- ▶ 用代码把隐含统计定义落到数据上；
- ▶ 从错误和中间结果中定位失败点；
- ▶ repair 后继续推进，而不是重启整题。

复现后看到的问题

- ▶ 上下文越长，越容易把旧日志、旧变量和最终答案混在一起；
- ▶ full harness 并非单调收益：会引入 verifier blocked、重复 debug 和分支选择错误；
- ▶ 多分支搜索如果没有选择准则，只是把错误候选变多；
- ▶ final block 若进入 code cell，会把答案格式机制反过来卡住 debug loop。

Takeaway: “时间换性能”不能理解成让小模型盲目多轮规划；多出来的时间必须服务于明确的检查、修复和停止条件。

Part II Definitions: 两档结构化方案

实验名	打开的机制	作用和风险
harness_light	DataCard / schema; Answer Spec; 基础技能清单; Final Block; 程序化格式器; 默认关闭重搜索和重 verifier。	轻量结构化约束: 先稳住列名、计算规则和输出格式。
harness_full	在 light 基础上增加语义 oracle / verifier, 检查统计方法、预处理顺序和格式; 必要时 targeted retry。	完整语义校验: 适合高风险题, 但会增加通信、阻断和分支选择成本。
hard-aware	调度原则, 不是独立实验名; 根据 Answer Spec、Verifier 失败类型和中间结果决定是否重算、换分支或升级模型。	hard 题不是简单加厚 prompt, 而是要能选择和复算。

关键词

DataCard 固定数据结构; Answer Spec 固定答案定义、计算规则和输出格式; Verifier 检查统计语义; Final Block 收束输出格式。light / full 不是模型大小, 也不是 easy / hard split, 而是同一 agent 上不同结构化强度的开关组合。

Takeaway: 第二部分比较的是结构化约束强度如何影响准确率、通信和失败风险。

Part II Scheme: 把长交互拆成可检查环节



执行前： 固定数据结构、答案规格、禁止动作。

执行后： 只在 verifier fail 后短上下文修复；复杂题才启用多候选执行与分支选择，不把完整日志反复交给模型。

Takeaway: 这条线的核心不是模块数量，而是每一轮交互都有输入、检查和停止条件。

Part II Link to Part III: 结构化时间也是通信控制

Part II 工件	后续阶段需要的信号	对 Part III 的意义
DataCard / schema	列名、类型、缺失值、候选日期列、ID 列	同一 CSV 可复用，减少反复读表探索。
Answer Spec	答案名、过滤、分组、统计方法、小数位、禁止动作	把长 prompt 压成短规范，低风险题可一次完成。
Verifier report	失败检查项、失败位置、是否必须 retry	retry 只传失败点，不回传完整 notebook 日志。
Final Block	原始值、格式检查、来源变量	reformat 不再自由抽取答案，可用程序化格式器。

和第三部分的关系

Part II 决定 “哪些信息值得在轮次之间通信”；Part III 决定 “这次通信该由规则程序、8B/14B 还是 32B 处理”。资源优化不是单独换小模型，而是先把交互内容压成可检查工件。

Takeaway: 结构化时间把 notebook 长交互变成短 artifact 传递，这是后面 EqTok 和调用次数优化的前提。

Part II Result & Analysis

Setting	ABQ	Hard ABQ	Avg runtime
baseline + original reformat	48.94%	44.00%	30.24s
harness_light	51.57%	47.32%	37.03s
harness_full	52.83%	49.44%	43.79s

Setting 名称保留实验脚本名: harness_light= 轻量结构化约束, harness_full= 完整语义校验。

light-first

默认只加最关键的结构化约束和格式器, 让大多数题先稳定完成。

targeted retry

只有 verifier fail 或高风险题, 才追加修复、搜索或升级模型; retry 输入是失败检查项, 不是整段日志。

hard-aware

hard 题不能只加分支数量, 还要能做分支选择、语义复算和必要的模型升级。

Takeaway: 结果支持“结构化约束有效”; 但 full 是否划算要看通信、阻断和重试成本, 这正引出第三部分的资源调度。

Part III

Resource-Aware Scheduling

如何在不改变效率的前提下降低成本消耗

Part III Baseline: 用 EqTok 重设资源指标

等效 token

$$\text{EqTok} = T_{32B} + 0.5T_{14B} + 0.25T_{8B}$$

基于 qwen 官方平台定价 1: 2: 4 换算等价 token。

32B baseline 指标	数值
ABQ	81.74%
PASQ	86.21%
UASQ	86.43%
调用次数	1,621
EqTok	5.961M

Takeaway: 这一部分只讨论 32B / 14B / 8B 的资源方案，避免把强模型参考和资源路由候选混在一起。

Part III 方案 A：分阶段资源使用规则

角色 / 阶段	实际任务	推荐处理
数据结构画像	读取数据表文件、列名、数据类型、缺失值、数值范围	确定性程序处理
答案规格解析	抽取答案名、列名、格式、小数位、禁止操作	规则优先；有歧义时用 8B / 14B
任务路由	判断概念、难度、风险等级	规则优先；有歧义时用 8B / 14B
步骤规划	拆分步骤、决定处理顺序	简单题用模板 / 8B；中等题用 14B；困难题用 32B
代码生成	编写数据处理、统计检验和机器学习代码	简单题用 14B；中等 / 困难题用 32B
调试执行	修复运行错误、列名错误、导入错误	简单题用 8B / 14B；中等题用 14B；困难题用 32B
调试后清理	清理错误单元，保留可运行路径	确定性程序处理
过程校验	检查列、样本数、方法规则、最终答案块	确定性程序处理
最终格式化	渲染规定答题格式	确定性程序处理

Takeaway: 模型路由不是为了全程用小模型，而是把强模型留给真的需要强推理的阶段。

Part III 方案 B：减少模型对话发生

机制	作用	影响
确定性工件	数据结构画像、答案规格、最终格式化 尽量程序化，不反复询问模型。	降低调用次数
规格到操作路径	从答案规格直接生成操作计划，减少自由探索。	降低资源消耗和偏题
一次性执行	低风险题一次模型调用完成。	降低时间
校验触发重试	失败才短上下文修复，不把完整日志再喂回去。	控制重试成本
工件复用	同一数据文件复用数据结构画像和列名规范化。	降低重复探索

注：第一行与上一页“数据结构画像 / 答案规格解析 / 过程校验 / 最终格式化”的确定性处理本质相同；这里强调它对减少模型对话的作用。

Takeaway: 这比狭义解题器更通用：目标不是覆盖固定统计公式，而是减少不必要的模型对话。

Part III Result: 资源消融真实结果

方案	ABQ	EqTok	调用次数
32B baseline	81.74%	5.960M	1,621
小模型规则	78.42%	3.517M	1,956
小模型规则 + 对话减少	78.63%	3.298M	1,874

强模型不一定更慢

强模型可能减少 debug / repair 轮数，所以总时间可能接近甚至更短。

小模型不一定更划算

小模型只有在简单、明确、短上下文子任务上才适合。

核心是风险调度

low risk 用 deterministic / small instruct; high risk 才用 32B thinking.

Final Integration

Resource-Aware DatawiseAgent

从三个局部改进到一个统一系统

Overview Architecture Figure

预留总览图位置：任务路由 / 结构化验证 / 上下文记忆 / 资源调度

建议放入 draw_overview 风格的轻量工程结构图，避免与前面单页机制重复。

感谢聆听